

Module 1: Characters, ASCII, Character Arrays, and Strings

1. Big Idea of This Module

In C/C++, strings are not magical objects. A string is made from characters stored in a character array, and it must end with a special character called the null terminator: `\0`.

The most important idea:

```
A C-style string = characters + '\0' at the end
```

Example:

```
"John" is stored as:  
J  o  h  n  \0
```

The `\0` tells C where the string stops.

2. Computers Store Characters as Numbers

A computer does not directly store letters like `A`, `B`, or `z`.

It stores numbers.

For example:

```
'A' is stored as 65  
'B' is stored as 66  
'a' is stored as 97  
'0' is stored as 48
```

These number values come from ASCII.

3. What Is ASCII?

ASCII is a character code system.

It maps characters to numbers.

Think of ASCII like a table or dictionary:

```
65 -> A
66 -> B
67 -> C
...
97 -> a
98 -> b
99 -> c
```

So when you write:

```
char temp = 'A';
```

C stores the number `65` inside `temp`.

4. Important ASCII Ranges

You should remember these ranges:

```
A to Z   -> 65 to 90
a to z   -> 97 to 122
0 to 9   -> 48 to 57
```

These ranges are useful later when checking whether a character is uppercase, lowercase, or a digit.

Example:

```
if (ch >= 65 && ch <= 90) {
    printf("Uppercase letter");
}
```

This checks whether `ch` is between `A` and `Z`.

5. Character `'0'` vs Number `0`

The character `'0'` is not the same as the number `0`.

```
char x = '0';  
printf("%d", x);
```

Output:

48

Why?

Because the character `'0'` has ASCII value `48`.

So:

```
0      -> number zero  
'0'    -> character zero, ASCII value 48
```

6. What Is `char` ?

A `char` is a data type used to store one character.

Example:

```
char letter = 'A';  
char digit = '5';  
char symbol = '#';
```

Each `char` stores one character value.

Internally, that character is stored as a number.

7. Single Quotes vs Double Quotes

Use single quotes for a single character.

```
'A'  
'x'
```

```
'7'  
'#'
```

Use double quotes for a string.

```
"A"  
"Hello"  
"John"
```

Important difference:

```
'A'    // one character  
"A"    // a string containing A and \0
```

So:

```
'A'  has 1 character  
"A"  has 2 characters: A and \0
```

8. Printing a Character as a Character or a Number

A `char` stores a number internally.

You can print it as a character using `%c`.

You can print it as a number using `%d`.

Example:

```
#include <stdio.h>  
  
int main() {  
    char temp = 'A';  
  
    printf("%c\n", temp);  
    printf("%d\n", temp);  
  
    return 0;  
}
```

Output:

```
A
65
```

Explanation:

```
%c means: show the value as a character
%d means: show the value as a number
```

The stored value is the same. Only the way we display it changes.

9. Memory Dry Run for `char temp = 'A'`

Code:

```
char temp = 'A';
```

What happens:

```
'A' has ASCII value 65.
So temp stores 65.
```

Then:

```
printf("%c", temp);
```

prints:

```
A
```

because `%c` converts the number `65` back into the character `A`.

But:

```
printf("%d", temp);
```

prints:

65

because `%d` shows the numeric value.

10. Character Arrays

A character array stores many characters.

Example:

```
char X[5] = {'A', 'B', 'C', 'D', 'E'};
```

Memory view:

Index:	0	1	2	3	4
Value:	A	B	C	D	E

This array has 5 characters.

11. Different Ways to Create Character Arrays

Method 1: Give the size manually

```
char X[5] = {'A', 'B', 'C', 'D', 'E'};
```

Here, the size is clearly given as `5`.

Method 2: Let the compiler find the size

```
char Y[] = {'A', 'B', 'C', 'D', 'E'};
```

Here, the compiler counts the values and creates an array of size `5`.

Method 3: Use ASCII numbers

```
char Z[5] = {65, 66, 67, 68, 69};
```

This also stores:

```
A B C D E
```

because:

```
65 -> A  
66 -> B  
67 -> C  
68 -> D  
69 -> E
```

12. Character Array vs String

A character array is not always a string.

A valid C-style string must end with `\0`.

Character array only

```
char arr[4] = {'J', 'o', 'h', 'n'};
```

Memory:

```
J   o   h   n
```

This is only a character array.

It is not a safe C-style string because it does not end with `\0`.

Valid C-style string

```
char name[5] = {'J', 'o', 'h', 'n', '\0'};
```

Memory:

```
J   o   h   n   \0
```

This is a valid C-style string.

13. What Is the Null Terminator `\0`?

The null terminator is a special character used to mark the end of a string.

It is written as:

```
'\0'
```

It tells C:

```
The string ends here.
```

Without `\0`, C does not know where the string ends.

14. Why `\0` Is Important

Consider this string:

```
char name[5] = {'J', 'o', 'h', 'n', '\0'};  
printf("%s", name);
```

Dry run:

```
index 0 -> J, print it  
index 1 -> o, print it  
index 2 -> h, print it  
index 3 -> n, print it  
index 4 -> \0, stop
```

Output:

John

`printf("%s", name)` keeps printing until it finds `\0`.

If `\0` is missing, C may continue reading memory after the array. This can produce garbage output or unsafe behavior.

15. Creating Strings Using Double Quotes

The easiest way to create a string is to use double quotes.

Instead of writing:

```
char name[5] = {'J', 'o', 'h', 'n', '\0'};
```

we can write:

```
char name[5] = "John";
```

C automatically adds `\0` at the end.

So this:

```
char name[5] = "John";
```

is stored as:

```
J   o   h   n   \0
```

16. String Size: Visible Characters + 1

A C-style string needs one extra space for `\0`.

Example:

```
char name[5] = "John";
```

John has 4 visible characters:

J o h n

But the string also needs `\0`:

J o h n \0

So "John" needs 5 spaces.

Rule:

Required size = number of visible characters + 1

Examples:

"A" needs 2 spaces: A \0
"Code" needs 5 spaces: C o d e \0
"Hello" needs 6 spaces: H e l l o \0

17. Partial Initialization of Character Arrays

If you create an array with fewer values than its size, C fills the remaining places with `0`.

Example:

```
char X[5] = {'A', 'B'};
```

Memory:

Index:	0	1	2	3	4
Value:	A	B	0	0	0

For characters, numeric `0` is the same value used by `\0`.

So you can also think of it as:

```
A  B  \0  \0  \0
```

This means the array can behave like the string `"AB"` if printed with `%s`.

18. Printing Characters and Strings

Print one character

Use `%c`.

```
char letter = 'A';  
printf("%c", letter);
```

Output:

```
A
```

Print a string

Use `%s`.

```
char name[10] = "John";  
printf("%s", name);
```

Output:

```
John
```

`%s` prints characters one by one until it finds `\0`.

19. Full Example: Character and String Printing

```
#include <stdio.h>
```

```

int main() {
    char temp = 'A';

    printf("As character: %c\n", temp);
    printf("As ASCII code: %d\n", temp);

    char name[10] = "John";

    printf("Name: %s\n", name);

    return 0;
}

```

Output:

```

As character: A
As ASCII code: 65
Name: John

```

20. Reading Input Using `scanf`

You can use `scanf` to read a single word.

Example:

```

char name[20];

scanf("%s", name);
printf("%s", name);

```

If the input is:

```
Sangeeth
```

The array stores:

```
S   a   n   g   e   e   t   h   \0
```

`scanf("%s", name)` automatically adds `\0` after the word.

21. Limitation of `scanf("%s")`

`scanf("%s")` stops reading when it sees a space.

Example input:

Taj Mahal

Code:

```
char place[20];  
  
scanf("%s", place);  
printf("%s", place);
```

Output:

Taj

Why?

Because `scanf("%s")` reads only one word.

It stops at the first space.

So it stores:

T a j \0

It does not store `Mahal` .

22. Reading Multi-Word Input

Sometimes we need to read text with spaces.

Example:

Taj Mahal

Older C examples may use:

```
gets(name);
```

But `gets()` is unsafe and should not be used in real programs.

The safer option is `fgets()`.

Example:

```
#include <stdio.h>

int main() {
    char name[50];

    fgets(name, 50, stdin);

    printf("%s", name);

    return 0;
}
```

If the input is:

```
Taj Mahal
```

`fgets()` can store the full line, including the space.

23. `scanf`, `gets`, and `fgets` Comparison

Function	Reads spaces?	Safe?	Notes
<code>scanf("%s", name)</code>	No	Safer than <code>gets</code> , but still needs care	Reads only one word
<code>gets(name)</code>	Yes	No	Unsafe; avoid it
<code>fgets(name, size, stdin)</code>	Yes	Yes, when size is correct	Best choice for full-line input

24. Mental Model for C-Style Strings

Think of a C-style string like a train.

```
J -> o -> h -> n -> \0
```

The letters are the passengers.

The `\0` is the final station.

When C prints a string using `%s`, it keeps moving through the array until it reaches `\0`.

If there is no final station, C does not know where to stop.

25. Complexity Notes

Character arrays

If an array stores `N` characters, it uses:

```
O(N) space
```

because more characters need more memory.

Printing a string

Printing a string takes:

```
O(N) time
```

because C prints characters one by one until it reaches `\0`.

Reading a string

Reading a string also takes:

```
O(N) time
```

because the input is read character by character.

26. Common Mistakes

Mistake 1: Forgetting space for `\0`

Wrong:

```
char name[4] = "John";
```

Better:

```
char name[5] = "John";
```

Reason:

```
John has 4 visible characters, but it needs 1 extra space for \0.
```

Mistake 2: Thinking `'0'` equals `0`

Wrong idea:

```
'0' is the same as 0
```

Correct idea:

```
'0' has ASCII value 48
```

Mistake 3: Using `%s` on a character array without `\0`

Unsafe:

```
char arr[4] = {'J', 'o', 'h', 'n'};  
printf("%s", arr);
```


Reason:

There is no `\0`, so `printf` may not know where to stop.

Mistake 4: Expecting `scanf("%s")` to read a full sentence

Input:

Taj Mahal

Code:

```
scanf("%s", place);
```

Only this is read:

Taj

Reason:

`scanf("%s")` stops at spaces.

Mistake 5: Using `gets()`

Avoid:

```
gets(name);
```

Use:

```
fgets(name, 50, stdin);
```

Reason:

gets() is unsafe because it does not know the array size.
fgets() is safer because we provide the maximum size.

27. Key Code Patterns

Print character and ASCII value

```
char temp = 'A';  
  
printf("%c\n", temp);  
printf("%d\n", temp);
```

Create character arrays

```
char X[5] = {'A', 'B', 'C', 'D', 'E'};  
char Y[] = {'A', 'B', 'C', 'D', 'E'};  
char Z[5] = {65, 66, 67, 68, 69};
```

Create a C-style string manually

```
char name[5] = {'J', 'o', 'h', 'n', '\0'};
```

Create a C-style string using double quotes

```
char name[5] = "John";
```

Print a string

```
printf("%s", name);
```

Read one word

```
scanf("%s", name);
```

Read a full line safely

```
fgets(name, 50, stdin);
```

28. Required Dry Runs

Dry Run 1: `temp` stores `A`

Code:

```
char temp = 'A';  
printf("%c", temp);  
printf("%d", temp);
```

Steps:

1. 'A' has ASCII value 65.
2. temp stores 65.
3. printf("%c", temp) prints A.
4. printf("%d", temp) prints 65.

Dry Run 2: Printing `"John"`

Code:

```
char name[5] = "John";  
printf("%s", name);
```

Memory:

Index:	0	1	2	3	4
Value:	J	o	h	n	\0

Steps:

1. Start at index 0: print J.
2. Move to index 1: print o.
3. Move to index 2: print h.
4. Move to index 3: print n.
5. Move to index 4: find \0.
6. Stop printing.

Output:

John

Dry Run 3: `scanf` with spaces

Code:

```
char place[20];
scanf("%s", place);
printf("%s", place);
```

Input:

Taj Mahal

Steps:

1. `scanf` starts reading from T.
2. It reads T, a, j.
3. It sees a space.
4. It stops reading.
5. It adds \0 after Taj.

Stored value:

```
T  a  j  \0
```

Output:

```
Taj
```

29. Practice Questions

Question 1

Print the ASCII values of:

```
A, Z, a, z, 0, 9
```

Expected values:

```
A -> 65  
Z -> 90  
a -> 97  
z -> 122  
0 -> 48  
9 -> 57
```

Question 2

How many spaces are needed for this string?

```
"Code"
```

Answer:

```
5 spaces
```

Reason:

```
C o d e \0
```

Question 3

Create the same array in two ways.

Using characters:

```
char A[5] = {'A', 'B', 'C', 'D', 'E'};
```

Using ASCII values:

```
char B[5] = {65, 66, 67, 68, 69};
```

Both store:

```
A B C D E
```

Question 4

What will this program print?

```
char place[20];  
scanf("%s", place);  
printf("%s", place);
```

Input:

```
Taj Mahal
```

Answer:

```
Taj
```

Reason:

`scanf("%s")` stops at the first space.

30. Final Module 1 Summary

Remember these points:

1. Computers store characters as numbers.
2. ASCII maps characters to numeric codes.
3. A char stores one character.
4. A character can be printed as a character using `%c`.
5. A character can be printed as a number using `%d`.
6. Character arrays store multiple characters.
7. A C-style string is a character array ending with `\0`.
8. The null terminator `\0` marks the end of the string.
9. Double-quoted strings automatically include `\0`.
10. A string needs one extra space for `\0`.
11. `printf("%s")` prints until it finds `\0`.
12. `scanf("%s")` reads only one word.
13. `scanf("%s")` stops at spaces.
14. `gets()` is unsafe and should be avoided.
15. `fgets()` is safer for reading multi-word input.

Final takeaway:

In C, a string is not just letters.
It is letters stored in a character array, followed by `\0`.